

MERN Stack Developer Assessment

Enterprise EMR Appointment Management System

Objective

Design, develop, and document a production-ready Appointment Management module for an Enterprise Electronic Medical Record (EMR) system using the MERN stack.

This assessment is designed to evaluate not only your ability to implement features but also your engineering mindset, software architecture, scalability, maintainability, security, database design, API design, and production readiness.

The quality of your implementation, code organization, and technical decisions is significantly more important than the number of features completed. If you are unable to complete every feature within the given time, prioritize writing clean, maintainable, and well-documented code rather than implementing incomplete or poorly structured functionality.

Technology Stack

Backend

- Node.js
- Express.js
- MongoDB
- Mongoose

Frontend

- React.js
- React Router

You may use any state management solution (Context API, Redux Toolkit, Zustand, React Query, etc.). Be prepared to justify your choice.

Functional Requirements

1. Authentication

Implement a secure authentication system.

Requirements:

- User Login
 - JWT Access Token
 - JWT Refresh Token
 - Password hashing using bcrypt
 - Protected APIs
 - Refresh Token handling
 - Logout with Refresh Token invalidation
-

2. Role-Based Access Control (RBAC)

Implement the following system roles.

Super Admin

- Create Doctors
- Create Receptionists
- Manage Doctor Schedules
- View all appointments
- Access all dashboards

Receptionist

- Search Patients
- Book Appointments
- Update Appointments
- Mark Patient as Arrived

Doctor

- View only their own appointments
- View patient information
- Update consultation notes (Optional)

All APIs must be protected using appropriate authorization middleware.

3. Doctor Schedule Management

Allow Super Admin to configure doctor schedules.

Each doctor should support:

- Working Days
- Multiple sessions per day
- Slot Duration
- Break Timings

Example

Morning Session

09:00 – 12:00

Lunch Break

12:00 – 13:00

Evening Session

13:00 – 17:00

Appointment slots must never be generated during break periods.

4. Appointment Scheduler

Implement an appointment scheduler.

Support filtering by:

- Doctor
- Department
- Date

Display:

- Available Slots
- Booked Slots

Prevent booking for:

- Past dates
- Past time slots
- Break periods

5. Dynamic Slot Generation

Generate appointment slots dynamically based on:

- Working hours
- Session timings
- Slot duration

Example:

Working Hours

09:00–10:00

Slot Duration

15 Minutes

Generated Slots

- 09:00
- 09:15
- 09:30
- 09:45

Slots must never overlap.

6. Appointment Booking

Support two patient types.

Existing Patient

Search by:

- Patient ID
- Mobile Number
- Name

New Patient

Automatically create a patient record.

Every appointment must be linked with:

- Patient
 - Doctor
 - Department
 - Appointment Slot
-

7. Concurrency Handling

Prevent double booking of appointment slots.

If two users attempt to book the same slot simultaneously, only one booking should succeed.

Implement an appropriate backend strategy such as atomic operations, transactions, unique constraints, or another reliable solution.

Your implementation should guarantee data consistency.

8. Appointment Management

Support:

- View Appointments
- Update Purpose
- Update Notes
- Cancel Appointment
- Mark Patient as Arrived

Maintain a proper appointment status workflow.

Example

Scheduled

↓

Arrived

↓

Completed

or

Cancelled

9. Search, Filtering & Pagination

Appointments should support:

- Patient Search
- Doctor Search
- Mobile Search
- Department Filter
- Status Filter
- Date Range

Implement:

- Server-side pagination
- Server-side sorting
- Server-side filtering

Avoid unnecessary database queries.

10. Logging & Audit Trail

Maintain an audit log for important actions.

Examples:

- Login
- Appointment Created
- Appointment Updated
- Appointment Cancelled

Each log should contain:

- User
 - Role
 - Action
 - Entity
 - Timestamp
-

Required API Endpoints

Authentication

- POST /api/v1/auth/login
- POST /api/v1/auth/logout
- POST /api/v1/auth/refresh

Doctors

- GET /api/v1/doctors

Slots

- GET /api/v1/slots

Appointments

- POST /api/v1/appointments
- GET /api/v1/appointments
- PUT /api/v1/appointments/
- DELETE /api/v1/appointments/
- POST /api/v1/appointments//arrive

Additional endpoints may be added if required.

Frontend Requirements

Develop responsive interfaces for:

- Login
- Dashboard
- Appointment Scheduler
- Appointment Booking
- Appointment List

The application should include:

- Calendar / Date Picker
- Slot Grid
- Form Validation
- Loading States
- Error Handling
- Responsive Design

Reusable components are expected wherever appropriate.

Security Requirements

Implement:

- Password Hashing
- JWT Authentication
- RBAC Middleware
- Input Validation
- Environment Variables
- Proper HTTP Status Codes
- Secure Error Responses
- CORS Configuration

Never expose sensitive information through API responses.

Performance Requirements

Optimize both frontend and backend.

Examples include:

- Efficient MongoDB Queries
- Proper Database Indexing
- Pagination
- Optimized API Responses
- Avoid unnecessary React re-renders
- Component memoization where appropriate

Document every optimization you implemented and explain why it was necessary.

Project Structure

The project should follow a clean and maintainable architecture.

Suggested backend structure:

- config

- controllers
- middlewares
- models
- routes
- services
- validators
- utils

Business logic should remain inside services.

Controllers should remain lightweight.

Avoid duplicated code.

Write reusable and modular components wherever possible.

Database Design

Design a scalable MongoDB database.

Document:

- Collections
- Relationships
- Indexes
- Query Optimization Strategy

Include sample documents or a simple database diagram explaining the relationships.

API Standards

All APIs should follow a consistent response structure.

Example

```
{  
  "success": true,  
  "message": "Appointment created successfully",  
  "data": {},  
  "meta": {}  
}
```

Use appropriate HTTP status codes throughout the application.

Engineering Challenge

Implement **Real-Time Appointment Updates** using WebSockets (Socket.IO or an equivalent solution).

When an appointment is:

- Created
- Updated
- Cancelled

all connected users viewing the scheduler should receive live updates without refreshing the page.

Include a brief explanation describing your implementation approach.

Production Readiness

Your project should be structured as though it will be deployed to production.

Use:

- Environment Variables
- Modular Folder Structure
- Centralized Error Handling
- Consistent API Responses
- Proper Configuration Management

Docker is optional and will be considered a bonus.

Engineering Decisions (Mandatory)

Create a document named:

`ENGINEERING_DECISIONS.md`

Briefly explain the reasoning behind the following:

- Why did you choose your project architecture?
- How did you design your MongoDB schema?
- How did you prevent double booking?
- Which database indexes did you create and why?
- What security measures did you implement?
- What performance optimizations did you apply?
- If this application needed to support millions of appointments, what architectural changes would you make?

The objective is to understand your engineering thought process rather than simply your implementation.

Code Quality Expectations

Your submission will be reviewed for overall engineering quality.

Please ensure that your project demonstrates:

- Clean and readable code
- Consistent naming conventions
- Proper separation of concerns
- Reusable components and utilities
- Minimal code duplication
- Modular architecture
- Meaningful commit history
- Proper error handling
- Maintainable folder structure

A smaller, well-structured solution is preferred over a larger solution with poor architecture.

Documentation

Include a README containing:

- Project Overview
- Folder Structure
- Architecture Overview

- Database Design
 - API Documentation
 - Environment Variables
 - Installation Instructions
 - Running the Project
 - Assumptions Made
 - Known Limitations
 - Future Improvements
-

Submission Requirements

Submit the following:

- Git Repository Link
- README
- `ENGINEERING_DECISIONS.md`
- API Documentation
- Database Schema or Diagram

Hosting the application is optional. If hosted, include the deployment URL in the README.

Notes

- Focus on writing production-quality code rather than implementing every possible feature.
- Clean architecture, maintainability, scalability, and code quality will carry more weight than the total number of completed features.
- If assumptions are made, document them clearly.
- You may use third-party libraries where appropriate, but you should be able to justify why they were chosen.
- Your submission should reflect how you would build and organize a real-world production application.